REFERENCE

Gatitas Tapabaños Nacatlán

Última actualización: 2026-04-10

Índice

1. Setup

- To print n decimals: `cout << fixed << setprecision(n);`

Bash script for compilation and execution

setup/compilation.sh

```bash
co() {
  g++ -std=c++20 -O2 -Wall -Wextra -Wshadow -Wconversion -o $1 $1.cpp
}

run() {
  co $1 && ./$1;
}
```

C++ template

setup/template.h

```cpp
#include <bits/stdc++.h>

using namespace std;

#define endl '\n'
#define dbg(x) cout << #x << ": " << x << endl
#define sz(x) int((x).size())
#define all(v) v.begin(), v.end()
#define rall(v) v.rbegin(), v.rend()
#define pb push_back
#define mp make_pair
#define fi first
#define se second

using ll = long long;
const ll INF = 1e18;
const ll MOD = 1e9 + 7;
```

2. Estructuras de Datos

2.1. Prefix Sums

2.1.1. 2D Prefix Sums

To process Q queries for the sum over a subrectangle of a 2D matrix with N rows and M columns

$$\textbf{prefix}[a][b] = \sum_{i=1}^{a} \sum_{j=1}^{b} \textbf{arr}[i][j]$$

This can be calculated as follows for row index $1 \le i \le n$ and column index $1 \le j \le m$:

$$\textbf{prefix}[i][j] = \textbf{prefix}[i-1][j] + \textbf{prefix}[i][j-1] - \textbf{prefix}[i-1][j-1] + \textbf{arr}[i][j]$$

The submatrix sum between rows a and A and columns b and B, can thus be expressed as follows:

$$\sum_{i=a}^{A} \sum_{j=b}^{B} \textbf{arr}[i][j] = \textbf{prefix}[A][B] - \textbf{prefix}[a-1][B] - \textbf{prefix}[A][b-1] + \textbf{prefix}[a-1][b-1]$$

2.2. Segment tree

Implementation

TIME
- $O(n)$ for building
- $O(\log n)$ for queries and updates

estructuras/segment-tree.cpp

```cpp
int n, t[4*MAXN];

void build(int a[], int v, int tl, int tr) {
  if (tl == tr) {
    t[v] = a[tl];
```

```cpp
    } else {
        int tm = (tl + tr) / 2;
        build(a, v*2, tl, tm);
        build(a, v*2+1, tm+1, tr);
        t[v] = t[v*2] + t[v*2+1];
    }
}

int sum(int v, int tl, int tr, int l, int r) {
    if (l > r)
        return 0;
    if (l == tl && r == tr) {
        return t[v];
    }
    int tm = (tl + tr) / 2;
    return sum(v*2, tl, tm, l, min(r, tm))
           + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
}

void update(int v, int tl, int tr, int pos, int new_val) {
    if (tl == tr) {
        t[v] = new_val;
    } else {
        int tm = (tl + tr) / 2;
        if (pos <= tm)
            update(v*2, tl, tm, pos, new_val);
        else
            update(v*2+1, tm+1, tr, pos, new_val);
        t[v] = t[v*2] + t[v*2+1];
    }
}
```

3. Ordenamiento y Búsqueda

3.1. Two Pointers

Some uses:

Subarray sum: Given an array of n positive integers and a target sum x, and we want to find a subarray whose sum is x or report that there is no such subarray.

2SUM Problem: Given an array of n numbers and a target sum x, find two array values such that their sum is x, or report that no such values exist.

Merging two sorted arrays: Given two arrays, sorted in ascending order combine the elements of these arrays into one big array.

```
while i < a.size() || j < b.size():
  if a[i] < b[j]:
    c[i + j] = a[i]
    i++
  else:
    c[i + j] = b[j]
    j++
```

Number of Smaller: We have two arrays a and b. We want to calculate for each element b_j how many such i exist that $a_i < b_j$.

```
i = 0
for j = 0..b.size()-1:
  while i < a.size() && a[i] < b[j]:
    i++
  res[j] = i
```

3.2. Binary Search on Monotonic Functions

Finding The Maximum x Such That $f(x) = \text{true}$

TIME
$O(\log n)$

ordenamiento_busqueda/last_true.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

int last_true(int lo, int hi, function<bool(int)> f) {
  // if none of the values in the range work, return lo - 1
  lo--;
  while (lo < hi) {
    // find the middle of the current range (rounding up)
    int mid = lo + (hi - lo + 1) / 2;
    if (f(mid)) {
      // if mid works, then all numbers smaller than mid also work
      lo = mid;
    } else {
      // if mid does not work, greater values would not work either
      hi = mid - 1;
    }
  }
```

```cpp
    return lo;
}

int main() {
    // all numbers satisfy the condition (outputs 10)
    cout << last_true(2, 10, [](int x) { return true; }) << endl;

    // outputs 5
    cout << last_true(2, 10, [](int x) { return x * x <= 30; }) << endl;

    // no numbers satisfy the condition (outputs 1)
    cout << last_true(2, 10, [](int x) { return false; }) << endl;
}
```

Finding The Minimum x Such That $f(x) = \text{true}$

TIME

$O(\log n)$

ordenamiento_busqueda/first_true.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

int first_true(int lo, int hi, function<bool(int)> f) {
    hi++;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (f(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}

int main() {
    // outputs 2
    cout << first_true(2, 10, [](int x) { return true; }) << endl;
    // outputs 6
    cout << first_true(2, 10, [](int x) { return x * x >= 30; }) << endl;
    // outputs 11
    cout << first_true(2, 10, [](int x) { return false; }) << endl;
}
```

4. Teoría de gráficas

4.1. Graph Traversal

4.1.1. Floodfill

Flood fill is an algorithm that identifies and labels the connected component that a particular cell belongs to in a multidimensional array.

> ⚠ **Advertencia**
>
> Recursive implementations of flood fill sometimes lead to
> - Memory limit exceeded if you run the recursive implementation on a really big grid (i.e., running the above code on a 4000×4000 grid may exceed 256 MB)
>   - Non-recursive implementations generally use less memory than recursive ones.

Example (recursive implementation)

graficas/floodfill_recursive.cpp

```cpp
const int MAX_N = 1000;

int grid[MAX_N][MAX_N];  // the grid itself
int row_num;
int col_num;
bool visited[MAX_N][MAX_N];  // keeps track of which nodes have been visited
int curr_size = 0;           // reset to 0 each time we start a new component

void floodfill(int r, int c, int color) {
    if ((r < 0 || r >= row_num || c < 0 || c >= col_num)  // if out of bounds
        || grid[r][c] != color                           // wrong color
        || visited[r][c]                                 // already visited this square
    )
        return;

    visited[r][c] = true;  // mark current square as visited
    curr_size++;           // increment the size for each square we visit

    // recursively call flood fill for neighboring squares
    floodfill(r, c + 1, color);
    floodfill(r, c - 1, color);
    floodfill(r - 1, c, color);
    floodfill(r + 1, c, color);
}
```

```cpp

int main() {
    /*
     * input code and other problem-specific stuff here
     */
    for (int i = 0; i < row_num; i++) {
        for (int j = 0; j < col_num; j++) {
            if (!visited[i][j]) {
                curr_size = 0;
                /*
                 * start a flood fill if the square hasn't already been visited,
                 * and then store or otherwise use the component size
                 * for whatever it's needed for
                 */
                floodfill(i, j, grid[i][j]);
            }
        }
    }
    return 0;
}
```

Example (iterative implementation)

graficas/floodfill_iterative.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

const int MAX_N = 2500;
const int R_CHANGE[]{0, 1, 0, -1};
const int C_CHANGE[]{1, 0, -1, 0};

int row_num;
int col_num;
string building[MAX_N];
bool visited[MAX_N][MAX_N];

void floodfill(int r, int c) {
    // Note: you can also use a queue and pop from the front for a BFS-based
    // approach
    stack<pair<int, int>> frontier;
    frontier.push({r, c});
    while (!frontier.empty()) {
        r = frontier.top().first;
        c = frontier.top().second;
        frontier.pop();

        if (r < 0 || r >= row_num || c < 0 || c >= col_num || building[r][c] == '#' ||
            visited[r][c])
            continue;

        visited[r][c] = true;
        for (int i = 0; i < 4; i++) {
            frontier.push({r + R_CHANGE[i], c + C_CHANGE[i]});
        }
    }
}

int main() {
    cin >> row_num >> col_num;
    for (int i = 0; i < row_num; i++) { cin >> building[i]; }

    int room_num = 0;
    for (int i = 0; i < row_num; i++) {
        for (int j = 0; j < col_num; j++) {
            if (building[i][j] == '.' && !visited[i][j]) {
                floodfill(i, j);
                room_num++;
            }
        }
    }
    cout << room_num << endl;
}
```

4.2. Disjoin Set Union

Find representative with path compression

TIME
$O(\log n)$

graficas/dsu_size.cpp

```cpp
    int find(int v) {
        if (v == parent[v])
            return v;
        return parent[v] = find(parent[v]);
    }
```

Union by size

TIME
$O(\alpha(m,n))$

graficas/dsu_size.cpp

```cpp
DSU_size(int n) : parent(n), size(n, 1) {
    for (int i = 0; i < n; i++) { parent[i] = i; }
}

bool unite(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b) return false;

    if (size[a] < size[b])
        swap(a, b);
    parent[b] = a;
    size[a] += size[b];

    return true;
}
```

Union by rank

TIME
$O(\alpha(m,n))$

graficas/dsu_rank.cpp

```cpp
DSU_rank(int n) : parent(n), rank(n, 0) {
    for (int i = 0; i < n; i++) { parent[i] = i; }
}

bool unite(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b) return false;

    if (rank[a] < rank[b])
        swap(a, b);
    parent[b] = a;
    if (rank[a] == rank[b])
        rank[a]++;

    return true;
}
```

4.3. Topological Sort

A topological sort of a directed acyclic graph is a linear ordering of its vertices such that for every directed edge $u \to v$ from vertex u to vertex v, u comes before v in the ordering.

DFS Version

graficas/topological_sort_dfs.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

vector<int> top_sort;
vector<vector<int>> graph;
vector<bool> visited;

void dfs(int node) {
    for (int next : graph[node]) {
        if (!visited[next]) {
            visited[next] = true;
            dfs(next);
        }
    }
    top_sort.push_back(node);
}

int main() {
    int n, m;  // The number of nodes and edges respectively
    cin >> n >> m;

    graph = vector<vector<int>>(n);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        graph[a - 1].push_back(b - 1);
    }

    visited = vector<bool>(n);
```

```cpp
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            visited[i] = true;
            dfs(i);
        }
    }
    reverse(top_sort.begin(), top_sort.end());

    vector<int> ind(n);
    for (int i = 0; i < n; i++) { ind[top_sort[i]] = i; }

    // Check if the topological sort is valid
    bool valid = true;
    for (int i = 0; i < n; i++) {
        for (int j : graph[i]) {
            if (ind[j] <= ind[i]) {
                valid = false;
                goto answer;
            }
        }
    }
answer:;

    if (valid) {
        for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 << ' '; }
        cout << top_sort.back() + 1 << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
}
```

BFS Version (Kahn's Algorithm)

graficas/topological_sort_bfs.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<vector<int>> graph(n);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        graph[a - 1].push_back(b - 1);
    }

    vector<int> in_degree(n);
    for (const vector<int> &nodes : graph) {
        for (int node : nodes) { in_degree[node]++; }
    }

    queue<int> queue;
    for (int i = 0; i < n; i++) {
        if (in_degree[i] == 0) { queue.push(i); }
    }
    vector<int> top_sort;
    while (!queue.empty()) {
        int curr = queue.front();
        queue.pop();
        top_sort.push_back(curr);
        for (int next : graph[curr]) {
            if (--in_degree[next] == 0) { queue.push(next); }
        }
    }

    if (top_sort.size() == n) {
        for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 << ' '; }
        cout << top_sort.back() + 1 << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
}
```

5. Programación Dinámica

Kadane's algorithm

TIME

$O(n)$

Given an array of integers, find the maximum sum subarray.

dp/kadane.cpp

```cpp
long long kadane(vector<long long>& arr) {
    long long max_ending_here = arr[0];
    long long max_so_far = arr[0];
    for (size_t i = 1; i < arr.size(); ++i) {
        max_ending_here = max(arr[i], max_ending_here + arr[i]);
        max_so_far = max(max_so_far, max_ending_here);
    }
    return max_so_far;
}
```

Knapsack 0/1

TIME
$O(nW)$

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

dp/knapsack01.cpp

```cpp
int knapsack01(vector<int>& w, vector<int>& v, int W) {
    int n = w.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));


    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j <= W; ++j) {
            dp[i][j] = dp[i-1][j]; // no tomar el objeto i
            if (w[i-1] <= j)
                dp[i][j] = max(dp[i][j], dp[i-1][j - w[i-1]] + v[i-1]);
        }
    }
    return dp[n][W];
}
```

Coin Change

dp/coin_change.cpp

```cpp
long long coin_change(vector<int>& coins, int X) {
    vector<long long> dp(X + 1, 0);
    dp[0] = 1; // una forma de formar 0

    for (int c : coins) {
        for (int x = c; x <= X; ++x) {
            dp[x] = (dp[x] + dp[x - c]) % MOD;
        }
    }
    return dp[X];
}
```

6. Teoría de números

6.1. Cribas

Criba de primos con optimizaciones básicas

TIME	SPACE
$O(n \log \log n)$	$O(n)$

Find all the prime numbers in $[1, n]$. This code uses the next optimizations:

- Sieving till root
- Sieving by the odd numbers only

teoria_de_numeros/sieve_with_basic_optimizations.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<bool> sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 3; i * i <= n; i += 2) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += 2 * i)
                is_prime[j] = false;
        }
    }
    return is_prime;
}
```

6.2. Euclid’s Algorithm

The algorithm is based on the formula

$$\gcd(a, b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

6.2.1. Extended Euclid's Algorithm:

Euclid's algorithm can also be extended so that it gives integers x and y for which

$$ax + by = \gcd(a, b)$$

Iterative version

TIME
$O(\log \min(a, b))$

teoria_de_numeros/extended_euclid_algorithm.cpp

```cpp
int gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}
```

6.3. Euler's Theorem

Euler's totient function $\varphi(n)$ gives the number of integers between $1, \ldots, n$ that are coprime to n.

Any value of $\varphi(n)$ can be calculated from the prime factorization of n using the formula

$$\varphi(n) = \prod_{i=1}^{k} p_i^{\alpha_i - 1}(p_i - 1)$$

Teorema 6.3.1 (Euler's theorem)

For all positive coprime integers x and m

$$x^{\varphi(m)} \bmod m = 1$$

6.4. Solving Equations

We can efficiently solve a Diophantine equation by using the extended Euclid's algorithm which gives integers x and y that satisfy the equation

$$ax + by = \gcd(a, b)$$

A Diophantine equation can be solved exactly when c is divisible by $\gcd(a, b)$.

If a pair (x, y) is a solution, then also all pairs

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right)$$

are solutions, where k is any integer.

7. Combinatoria

7.1. Binomial Coefficients

Nota

For $n < k$ the value of $\binom{n}{k}$ is assumed to be zero.

Basic Improved Implementation

```cpp
int C(int n, int k) {
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (int)(res + 0.01);
}
```

Table of binomial coefficients using Pascal's Triangle identity

```cpp
const int maxn = ...;
int C[maxn + 1][maxn + 1];
C[0][0] = 1;
for (int n = 1; n <= maxn; ++n) {
    C[n][0] = C[n][n] = 1;
    for (int k = 1; k < n; ++k)
        C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
}
```

8. Strings

8.1. Trie

Estructura

strings/trie.cpp

```cpp
const int K = 26;
```

```cpp

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);
```

Agregar una cadena

strings/trie.cpp

```cpp
void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}
```

hash-doble

strings/Hash_Doble.cpp

```cpp
struct Hash {//doble hashing
    vector<ll> h1, h2;
    vector<ll> p1, p2;
    ll p=31;//tomamos esta p para evitar colisiones
    Hash(string &s) {
        int n = s.size();
        h1.resize(n+1, 0);
        h2.resize(n+1, 0);
        p1.resize(n+1, 1);
        p2.resize(n+1, 1);
//hacemos doble hasheo para evitar colisiones
        for (int i = 0; i < n; i++) {
            p1[i+1] = (p1[i] * p) % MOD;//a uno le aplicamos modulo
            p2[i+1] = (p2[i] * p);//el otro ocupamos el modulo implicito de ll
//uno con mod 10e9+7 y otro con 2e64 que es LLONG_MAX
            int val = s[i] - 'a' + 1;//hasheamos con el asci

            h1[i+1] = (h1[i] * p + val) % MOD;
            h2[i+1] = (h2[i] * p + val);
        }
    }

    pair<ll,ll> get(ll l, ll r) {//comparamos los 2 hashing
        ll x1 = (h1[r+1] - h1[l] * p1[r-l+1] % MOD + MOD) % MOD;
        ll x2 = (h2[r+1] - h2[l] * p2[r-l+1]);
        return {x1, x2};
    }
};
Hash hs(s);//inicializar
hs.get(i,j);//lugar de sonde se compara
if(hs.get(l,m)==hs.get(i,j))//si 2 cadenas son iguales
```

KMP — Knuth-Morris-Pratt

TIME	SPACE
$O(n+m)$	$O(m)$

Cuenta ocurrencias de pat en txt. buildPhi construye el arreglo de fallos: phi[j] = prefijo propio más largo de pat[0..j] que es sufijo. Al fallar retrocede a phi[i-1] en lugar de reiniciar.

strings/kmp.cpp

```cpp
// snippet: kmp
vll buildPhi(const vll& pat) {
    ll m = pat.size();
    vll phi(m, 0);
    for (ll j = 1, i = 0; j < m; j++) {
        while (i > 0 && pat[i] != pat[j]) i = phi[i - 1];
        if (pat[i] == pat[j]) i++;
        phi[j] = i;
    }
    return phi;
}

ll KMP(const vll& pat, const vll& txt) {
    ll m = pat.size(), n = txt.size();
```

```
    if (m == 0) return 0;
    vll phi = buildPhi(pat);
    ll matches = 0;
    for (ll i = 0, j = 0; j < n; j++) {
        while (i > 0 && pat[i] != txt[j]) i = phi[i - 1];
        if (pat[i] == txt[j]) i++;
        if (i == m) { matches++; i = phi[i - 1]; }
    }
    return matches;
}
// snippet: end
```

9. Manipulación de bits

9.1. Máscara de bits

9.1.1. Manipulación individual de bits

Establece en 1 el k-ésimo bit de x:

```
#define ON(x, k) ((x) | (1LL << (k)))
```

Establece en 0 el k-ésimo bit de x:

```
#define OFF(x, k) ((x) & ~(1LL << (k)))
```

Invierte el k-ésimo bit de x:

```
#define TOGGLE(x, k) ((x) ^ (1LL << (k)))
```

Comprueba si el k-ésimo bit de x es 1:

```
#define CHECK(x, k) ((x) & (1LL << (k)))
```

9.1.2. Working with the rightmost 1-bit

x & -x: Isolates the rightmost set bit (keeps only the least significant bit that is 1, sets all others to 0).

x & (x - 1): Removes the rightmost set bit from x (turns the rightmost 1 to 0).

x | (x - 1): Sets all trailing zeros of x to 1.

9.1.3. Creating useful bit masks

(1 << n) - 1: Creates a bit mask with the first n bits set to 1.

9.1.4. Common bit manipulation tricks

- A positive number x is a power of two exactly when x & (x - 1) = 0.

x ^ A ^ B: Determines which is not the current value of x between A and B.

9.1.5. Bit operations as math alternatives

x << k: equivalent to x * pow(2, k)

x >> k: equivalent to x / pow(2, k) (integer division)

x & ((1 << k) - 1): equivalent to x % pow(2, k)

A + B: equivalent to (A ^ B) + 2 * (A & B) or (A | B) + (A & B)

9.1.6. GNU C++ compiler built-in functions

__builtin_clz(x): the number of leading zeros (zeros on the left side of the bit representation).

__builtin_ctz(x): the number of trailing zeros (zeros on the right side of the bit representation).

__builtin_popcount(x): the number of ones in the bit representation.

__builtin_parity(x): the parity (even or odd) of the number of ones in the bit representation.

Example of use

```
int x = 5328; // 00000000000000000001010011010000
cout << __builtin_clz(x) << "\n";     // 19
cout << __builtin_ctz(x) << "\n";     // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n";  // 1
```

> **Nota**
>
> The above functions only support int numbers, but there are also long long versions of the functions available with the suffix ll (e.g. __builtin_popcountll(x)).

9.2. Representing Sets

Every subset of a set $\{0, 1, 2, \ldots, n - 1\}$ can be represented as an n bit integer whose one bits indicate which elements belong to the subset.

Goes through the subsets with exactly k elements

```
for (int b = 0; b < (1<<n); b++) {
    if (__builtin_popcount(b) == k) {
        // process subset b
    }
}
```

Goes through the subsets of a set x

```cpp
int b = 0;
do {
    // process subset b
} while (b=(b-x)&x);
```

The idea is that the formula $b - x$ detects the rightmost one bit in x that is zero in b. This bit becomes one and all bits after it become zero. Then the and operation ensures that the resulting value is a subset of x. Note that $b - x$ equals $-(x - b)$ so we can think that we first remove all one bits that appear in b and then invert the value and add one.

9.3. Bitsets

The `bitset` structure corresponds to an array whose each value is either 0 or 1.

Creates a bitset of 10 elements

```cpp
bitset<10> s;
s[1] = 1;
s[3] = 1;
s[4] = 1;
s[7] = 1;
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

Returns the number of one bits in the bitset

```cpp
cout << s.count() << "\n"; // 4
```

Bit operations can be directly used to manipulate bitsets

```cpp
bitset<10> a, b;
// ...
bitset<10> c = a&b;
bitset<10> d = a|b;
bitset<10> e = a^b;
```

Recorrer bitset

```cpp
for (int i = 0; i < (int)bs.size(); ++i) {
    cout << "bit " << i << ": " << bs[i] << "\n";
}
```

Mostrar máscara como binario

```cpp
cout << "maskNum = " << bitset<8>(maskNum) << '\n';
```

10. Definiciones y resultados útiles

10.1. Teoría de números

10.1.1. Primos

- Primeros 50 números primos:
  2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229
- Para todo entero $n > 1$, existe una factorización prima única
  $$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$$
- La función contadora de primos $\pi(n)$ da el *número de primos hasta* n:
  $$\pi(n) \approx \frac{n}{\ln(n)}$$
- Número de divisores de un entero n:
  $$\tau(n) = \prod_{i=1}^{k} (\alpha_i + 1)$$
- Suma de divisores de un entero n:
  $$\sigma(n) = \prod_{i=1}^{k} (1 + p_i + \ldots + p_i^{\alpha_i}) = \prod_{i=1}^{k} \frac{p_i^{\alpha_i + 1} - 1}{p_i - 1}$$

10.1.2. Aritmética modular

Definición 10.1.1 (Modular multiplicative inverse)

The *modular multiplicative inverse* of x with respect to m is a value $\mathrm{inv}_m(x)$ such that

$$x \cdot \mathrm{inv}_m(x) \bmod m = 1$$

If m is prime

$$\mathrm{inv}_m(x) = x^{m-2}$$

- Factorial inverso modular:
  $\mathrm{inv}_p((x-1)!) \equiv x * \mathrm{inv}_p((x!)) \bmod p$

10.2. Combinatoria

10.2.1. Números de Fibonacci

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

- Primeros 30 términos:
  0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229

10.2.2. Coeficientes binomiales

- Pascal's Triangle:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

- Symmetry rule:

$$\binom{n}{k} = \binom{n}{n-k}$$

- Factoring in:

$$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

- Sum over k:

$$\sum_{k=0}^{n}\binom{n}{k} = 2^n$$

- Sum over n:

$$\sum_{m=0}^{n}\binom{m}{k} = \binom{n+1}{k+1}$$

- Sum over n and k:

$$\sum_{k=0}^{m}\binom{n+k}{k} = \binom{n+m+1}{m}$$

- Sum of the squares:

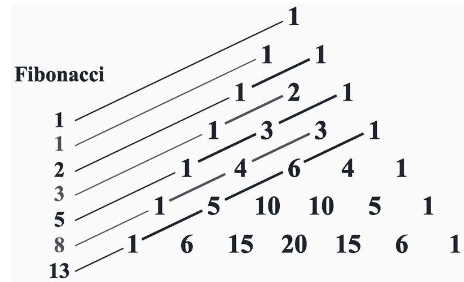
$$\binom{n}{0}^2 + \binom{n}{1}^2 + \cdots + \binom{n}{n}^2 = \binom{2n}{n}$$

- Weighted sum:

$$1\binom{n}{1} + 2\binom{n}{2} + \cdots + n\binom{n}{n} = n2^{n-1}$$

- Connection with the Fibonacci numbers:

$$\binom{n}{0} + \binom{n-1}{1} + \cdots + \binom{n-k}{k} + \cdots + \binom{0}{n} = F_{n+1}$$



10.2.3. Desarreglos

11. Utilidades

11.1. Strings

Method/Syntax	Description	Complexity	Example
s.insert(pos, str)	Inserts string at position	O(n+m)	s.insert(2, "abc");
s.erase(pos, len)	Removes len characters from pos	O(n)	s.erase(2, 3);
s.substr(pos, len)	Returns substring from pos with len chars	O(len)	string sub = s.substr(2, 5);
s.find(str, pos)	Finds first occurrence of str from pos	O(n*m)	size_t pos = s.find("abc");
s.rfind(str, pos)	Finds last occurrence of str before pos	O(n*m)	size_t pos = s.rfind("abc");
s.find_first_of(chars)	Finds first occurrence of any char in chars	O(n*m)	s.find_first_of("aeiou");
s.find_last_of(chars)	Finds last occurrence of any char in chars	O(n*m)	s.find_last_of("aeiou");
s.find_first_not_of(chars)	Finds first char not in chars	O(n*m)	s.find_first_not_of(" \t");
s.find_last_not_of(chars)	Finds last char not in chars	O(n*m)	s.find_last_not_of(" \t");
s.replace(pos, len, str)	Replaces len chars at pos with str	O(n+m)	s.replace(2, 3, "xyz");
s.compare(str)	Lexicographically compares strings	O(min(n,m))	int cmp = s.compare("abc");
s.c_str()	Returns C-style null-terminated string	O(1)	const char* cstr = s.c_str();
s.data()	Returns pointer to character array	O(1)	char* ptr = s.data();
s.reserve(n)	Reserves capacity for n characters	O(1)	s.reserve(1000);
s.resize(n, c)	Changes size to n, filling with c if needed	O(n)	s.resize(10, 'x');
s.clear()	Removes all characters	O(n)	s.clear();
s.swap(other)	Swaps contents with another string	O(1)	s.swap(t);
getline(cin, s)	Reads entire line including spaces	O(n)	getline(cin, s);
getline(cin, s, delim)	Reads until delimiter character	O(n)	getline(cin, s, ',');
stoi(s) / stol(s) / stoll(s)	Converts string to int/long/long long	O(n)	int num = stoi("123");
stof(s) / stod(s)	Converts string to float/double	O(n)	double d = stod("3.14");
to_string(num)	Converts number to string	O(log num)	string s = to_string(42);
isalpha(c) / isdigit(c)	Checks if char is letter/digit	O(1)	if (isalpha(s[i])) { ... }
islower(c) / isupper(c)	Checks if char is lowercase/uppercase	O(1)	if (islower(s[i])) { ... }
tolower(c) / toupper(c)	Converts char to lowercase/uppercase	O(1)	s[i] = tolower(s[i]);
isspace(c)	Checks if char is whitespace	O(1)	if (isspace(s[i])) { ... }
isalnum(c)	Checks if char is alphanumeric	O(1)	if (isalnum(s[i])) { ... }

11.1.1. Important Notes:

- **String indexing**: Use `s[i]` for fast access, `s.at(i)` for bounds checking

- **Find functions**: Return string::npos if not found (usually −1 or large value)
- **Character functions**: isalpha, isdigit, etc. are in <cctype> header
- **Conversion functions**: stoi, to_string, etc. are in <string> header
- **Amortized complexity**: push_back is O(1) on average due to capacity doubling

11.1.2. Useful Patterns:

```cpp
// Check if substring exists
if (s.find("pattern") != string::npos) { ... }

// Convert string to lowercase
transform(s.begin(), s.end(), s.begin(), [](char c) { return tolower(c); });

// Split string by delimiter
stringstream ss(s); string token;
while (getline(ss, token, ',')) { ... }

// Remove leading/trailing whitespace
s.erase(0, s.find_first_not_of(" \t"));
s.erase(s.find_last_not_of(" \t") + 1);

// Reverse string
reverse(s.begin(), s.end());
```

11.2. STL Algorithm

Function/Syntax	Description	Complexity	Example
sort(begin, end)	Sorts elements in ascending order	O(n log n)	sort(v.begin(), v.end())
stable_sort(begin, end)	Sorts maintaining relative order of equal elements	O(n log n)	stable_sort(v.begin(), v.end())
reverse(begin, end)	Reverses the order of elements	O(n)	reverse(v.begin(), v.end())
binary_search(begin, end, val)	Searches for value in sorted range (returns bool)	O(log n)	binary_search(v.begin(), v.end(), x)
lower_bound(begin, end, val)	Finds first element ≥ val	O(log n)	lower_bound(v.begin(), v.end(), x)
upper_bound(begin, end, val)	Finds first element > val	O(log n)	upper_bound(v.begin(), v.end(), x)
equal_range(begin, end, val)	Returns pair of iterators [lower_bound, upper_bound]	O(log n)	equal_range(v.begin(), v.end(), x)
min_element(begin, end)	Finds iterator to minimum element	O(n)	min_element(v.begin(), v.end())
max_element(begin, end)	Finds iterator to maximum element	O(n)	max_element(v.begin(), v.end())
minmax_element(begin, end)	Returns pair {min_iter, max_iter}	O(n)	minmax_element(v.begin(), v.end())
next_permutation(begin, end)	Generates next lexicographic permutation	O(n)	next_permutation(v.begin(), v.end())
prev_permutation(begin, end)	Generates previous lexicographic permutation	O(n)	prev_permutation(v.begin(), v.end())
unique(begin, end)	Removes consecutive duplicate elements	O(n)	unique(v.begin(), v.end())

count(begin, end, val)	Counts occurrences of a value	O(n)	count(v.begin(), v.end(), x)
count_if(begin, end, pred)	Counts elements satisfying condition	O(n)	count_if(v.begin(), v.end(), [](int x){return x%2==0;})
find(begin, end, val)	Finds first occurrence of value	O(n)	find(v.begin(), v.end(), x)
find_if(begin, end, pred)	Finds first element satisfying condition	O(n)	find_if(v.begin(), v.end(), [](int x){return x>5;})
rotate(begin, middle, end)	Rotates elements (middle becomes new begin)	O(n)	rotate(v.begin(), v.begin()+k, v.end())
random_shuffle(begin, end)	Shuffles elements randomly	O(n)	random_shuffle(v.begin(), v.end())
shuffle(begin, end, rng)	Shuffles with specific random generator	O(n)	shuffle(v.begin(), v.end(), rng)
fill(begin, end, val)	Fills range with a value	O(n)	fill(v.begin(), v.end(), 0)
iota(begin, end, val)	Fills with consecutive values starting from val	O(n)	iota(v.begin(), v.end(), 1)
accumulate(begin, end, init)	Sums elements (requires <numeric>)	O(n)	accumulate(v.begin(), v.end(), 0)
partial_sum(begin, end, out)	Calculates partial sums	O(n)	partial_sum(v.begin(), v.end(), prefix.begin())
merge(begin1, end1, begin2, end2, out)	Merges two sorted ranges	O(n+m)	merge(a.begin(), a.end(), b.begin(), b.end(), c.begin())
inplace_merge(begin, middle, end)	Merges two sorted parts of same range	O(n log n)	inplace_merge(v.begin(), v.begin()+k, v.end())
set_union(begin1, end1, begin2, end2, out)	Union of two sorted sets	O(n+m)	set_union(a.begin(), a.end(), b.begin(), b.end(), c.begin())
set_intersection(begin1, end1, begin2, end2, out)	Intersection of two sorted sets	O(n+m)	set_intersection(a.begin(), a.end(), b.begin(), b.end(), c.begin())
set_difference(begin1, end1, begin2, end2, out)	Difference of two sorted sets	O(n+m)	set_difference(a.begin(), a.end(), b.begin(), b.end(), c.begin())
nth_element(begin, nth, end)	Partitions so nth element is in correct position	O(n) avg	nth_element(v.begin(), v.begin()+k, v.end())
partition(begin, end, pred)	Partitions elements by predicate	O(n)	partition(v.begin(), v.end(), [](int x){return x%2==0;})
is_sorted(begin, end)	Checks if range is sorted	O(n)	is_sorted(v.begin(), v.end())
is_permutation(begin1, end1, begin2)	Checks if one range is permutation of another	$O(n^2)$	is_permutation(a.begin(), a.end(), b.begin())

11.2.1. Important Notes:

- **Sorted ranges required**: binary_search, lower_bound, upper_bound, equal_range, merge, set_* require sorted ranges
- **<numeric> functions**: accumulate, partial_sum, iota are in <numeric>, not <algorithm>
- **Iterators**: Many functions return iterators. To get indices: distance(v.begin(), iter)
- **Predicates**: Functions like count_if, find_if accept lambdas or functors
- **Custom comparators**: Most accept custom comparators as third parameter

11.2.2. Useful Combinations:

```cpp
// Sort and remove duplicates
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
```

11.3. STL containers

11.3.1. Deques

A ***deque*** is a dynamic array that can be efficiently manipulated at both ends of the structure.

```cpp
deque<int> d;
d.push_back(5); // [5]
d.push_back(2); // [5,2]
d.push_front(3); // [3,5,2]
d.pop_back(); // [3,5]
d.pop_front(); // [5]
```

The operations of a deque also work in $O(1)$ average time. Deques should be used only if there is a need to manipulate both ends of the array.

11.3.2. Set Structures

The basic operations of sets are element insertion, search and removal. Sets are implemented so that all the above operations are efficient.

11.3.2.1. Sets and Multisets

Basic functions

- **insert** adds an element to the set
- **count** returns the number of occurrences of an element in the set
- **erase** removes an element from the set
- **find**(x) returns an iterator that points to an element whose value is x. However, if the set does not contain x, the iterator will be **end**().

Prints the number of elements in a set

```cpp
cout << s.size() << "\n";
for (auto x : s) {
    cout << x << "\n";
}
```

Use of find

```cpp
auto it = s.find(x);
if (it == s.end()) {
    // x is not found
}
```

Multisets: A *multiset* is a set that can have several copies of the same value. C++ has the structures **multiset** and **unordered_multiset**.

```cpp
multiset<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 3
```

The function erase removes all copies of a value from a multiset:

```cpp
s.erase(5);
cout << s.count(5) << "\n"; // 0
```

Often, only one value should be removed, which can be done as follows:

```cpp
s.erase(s.find(5));
cout << s.count(5) << "\n"; // 2
```

Note that the functions **count** and **erase** have an additional $O(k)$ factor where k is the number of elements counted/removed. In particular, it is *not* efficient to count the number of copies of a value in a multiset using the **count** function.

11.3.3. Priority Queues

Insertion and removal take $O(\log n)$ time, and retrieval takes $O(1)$ time.

If we only need to efficiently find minimum or maximum elements, it is a good idea to use a priority queue instead of a set or multiset.

By default, the elements in a C++ priority queue are sorted in decreasing order, and it is possible to find and remove the largest element in the queue.

```cpp
priority_queue<int> q;
q.push(3);
q.push(5);
q.push(7);
q.push(2);
cout << q.top() << "\n"; // 7
q.pop();
cout << q.top() << "\n"; // 5
q.pop();
q.push(6);
cout << q.top() << "\n"; // 6
q.pop();
```

If we want to create a priority queue that supports finding and removing the smallest element, we can do it as follows:

```cpp
priority_queue<int,vector<int>,greater<int>> q;
```

11.3.4. Policy-based data structures

The g++ compiler also supports some data structures that are not part of the C++ standard library. Such structures are called policy-based data structures. To use these structures, the following lines must be added to the code:

```cpp
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

After this, we can define a data structure **indexed_set** that is like set but can be indexed like an array. The definition for int values is as follows:

```cpp
using indexed_set = tree<
  int,
  null_type,
```

```
    less<int>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;
```

This data structure supports all the operations as a set in C++ in addition to the following:

- find_by_order(k): returns an iterator to the k-th smallest element (0-based indexing)
- order_of_key(x): returns the number of elements in the set that are strictly less than x